

Relazione Parallel Computing

Traccia: Parallel Image Augmentation

Youness Aharram 7198778

Abstract

0 Questo lavoro affronta il problema dell'*image augmentation*, cioè l'applicazione di trasformazioni casuali a un dataset di immagini, formulato come un carico di lavoro fortemente parallelo basato sulla libreria **Albumentations**. L'obiettivo è valutare l'efficacia della parallelizzazione confrontando un'implementazione puramente sequenziale con una versione parallela. Lo studio ha compreso l'analisi di tre pipeline di costo computazionale crescente, diverse politiche di bilanciamento del carico (scheduling statico e dinamico), la variazione del chunk size e il trade-off tra potenza di calcolo e overhead di comunicazione inter-processo. È stato inoltre confrontato il modello a processi con quello a thread, ed è stata condotta un'analisi fisica della macchina per spiegare i limiti alla scalabilità. Il sistema parallelo ottimizzato ha dimostrato una buona scalabilità raggiungendo uno speedup massimo di **6.19×** rispetto alla versione sequenziale.

Introduzione

L'*image augmentation* è la fase di pre-processing che alimenta ogni pipeline moderna di computer vision: ogni immagine del dataset viene trasformata casualmente (rotazioni, variazioni di colore, rumore, distorsioni) per ampliare artificialmente la varietà dei dati di addestramento. È un'operazione intrinsecamente ripetitiva e, su dataset di dimensioni realistiche, molto costosa dal punto di vista computazionale. La motivazione principale di questo progetto è analizzare le strategie di parallelizzazione applicabili a questo carico di lavoro e confrontarle con le prestazioni ottenibili da una soluzione sequenziale, che sfrutta un solo core lasciando inutilizzato tutto il resto della CPU.

Dal punto di vista computazionale, ogni immagine viene elaborata in modo completamente indipendente dalle altre: non esistono dipendenze tra le iterazioni, non è necessaria alcuna sincronizzazione e non serve una sezione critica. Il costo di elaborazione di una singola immagine dipende dal numero e dal tipo di trasformazioni applicate, e può variare da poche centinaia di microsecondi a diversi millisecondi. Partendo da queste premesse, il progetto si pone diversi obiettivi. Il primo passo è l'implementazione pratica: verrà creata un'architettura parallela e una versione puramente sequenziale che servirà per effettuare i confronti. Successivamente si passerà alla misurazione delle prestazioni reali del sistema, calcolando lo *Strong Scaling*, per osservare quanto il programma diventa più veloce all'aumentare del numero di core, il *Weak Scaling*, per verificare se il sistema mantiene la propria efficienza quando il carico di lavoro cresce proporzionalmente al numero di core, e il *Throughput*, per quantificare la potenza netta di elaborazione in termini di immagini al secondo. Infine, l'ultimo obiettivo è analizzare i classici colli di bottiglia della programmazione parallela: verrà studiata la dimensione ottimale dei pacchetti di lavoro (*chunk*), verrà misurato l'overhead dovuto alla comunicazione tra processi (*IPC*) e verrà spiegato, misurandolo direttamente sull'hardware, perché lo speedup si ferma al di sotto

del valore ideale.

Implementazione sequenziale

L'implementazione sequenziale costituisce la baseline dell'intero progetto. La valutazione delle sue prestazioni è fondamentale, poiché fornisce il tempo di riferimento assoluto T_1 necessario per il calcolo dello *speedup*. L'algoritmo riceve in input un dataset di immagini e vi applica, una alla volta e in un unico processo, la pipeline di trasformazioni di Albumentations, misurando il tempo totale di esecuzione con un timer. Il dataset è **sintetico ma procedurale e deterministico**: l'immagine di indice i viene generata da un generatore pseudo-casuale inizializzato con un seed derivato dall'indice stesso, sovrapponendo un gradiente di fondo, una texture a bassa frequenza, alcune forme geometriche con bordi netti e grana fotografica. Questo approccio evita di dover scaricare gigabyte di dati e rende il dataset ricostruibile bit per bit su qualunque macchina, così che il numero di immagini, la loro dimensione e la complessità della pipeline diventino parametri liberi dell'esperimento. La configurazione principale usa 3000 immagini 512×512 (circa 2.2 GB in memoria), ridotte in uscita a 256×256 .

Per studiare come la scalabilità dipenda dal peso del calcolo sono state definite tre pipeline di costo crescente: una *light* (3 trasformazioni, circa 0.25 ms per immagine), una *medium* (7 trasformazioni, 1.11 ms) e una *heavy* (10 trasformazioni fra cui deformazioni elastiche, sfocature e rumore gaussiano, 6.39 ms).

Identificazione delle parti parallelizzabili

L'analisi del flusso di esecuzione della versione sequenziale evidenzia una proprietà fondamentale del problema: l'immagine augmentation può essere classificata come un caso di problema *imbarazzantemente parallelo*. In questo contesto, ogni operazione di calcolo risulta indipendente dalle altre. In particolare, la generazione della trasformazione, la sua applicazione all'immagine e la produzione del risultato non dipendono in alcun modo dalle immagini elaborate in precedenza o in parallelo. Non sono presenti stati condivisi in scrittura né dipendenze tra iterazioni successive che richiedano sincronizzazione o comunicazione intermedia per garantire la correttezza dell'algoritmo. Di conseguenza, il ciclo che itera sulle immagini rappresenta il punto naturale di parallelizzazione.

L'intero dataset può essere suddiviso in sottoinsiemi indipendenti, assegnando a ciascun processo un gruppo distinto di immagini da elaborare. Questa suddivisione in unità di lavoro (*chunk*) consente di distribuire il carico computazionale su più core, mantenendo una struttura semplice e con overhead minimo. L'unica risorsa realmente condivisa è il dataset di input, che però viene soltanto letto e mai modificato: questa immutabilità apre la possibilità di condividerlo tra i processi senza copie e senza alcuna protezione. Tale proprietà strutturale costituisce la base teorica per l'adozione di un modello di parallelizzazione di tipo *Map-Reduce*: nella fase di *map*, ogni processo elabora il proprio sottoinsieme di immagini e produce un risultato parziale; nella fase di *reduce*, i risultati parziali vengono aggregati per ottenere la soluzione finale.

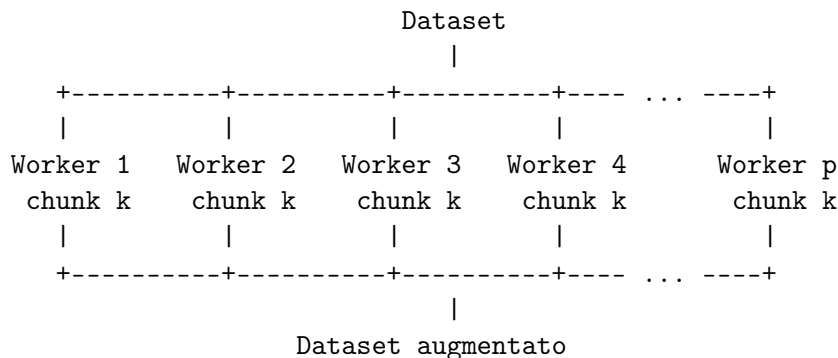
Implementazione parallela

L'implementazione parallela del sistema si basa su un modello di tipo *Map-Reduce*, sfruttando il modulo `multiprocessing` di Python. L'obiettivo è distribuire il carico computazionale su più processi, mantenendo al contempo una struttura semplice e modulare.

Architettura generale

L'algoritmo parallelo mantiene la stessa logica della versione sequenziale, ma suddivide il dataset in unità di lavoro indipendenti. In particolare, ogni task corrisponde a un gruppo di immagini identificate dai loro indici. Il dataset viene quindi trasformato in una lista di task, ciascuno dei quali viene assegnato a un processo del pool. La funzione principale `run_process_pool` si occupa di:

- costruire la lista dei task (gruppi di indici di immagini),
- configurare il grado di parallelismo (numero di worker),
- definire la strategia di scheduling,
- avviare l'esecuzione parallela,
- aggregare i risultati parziali (fase di *reduce*).



Fase di Map

La fase di *map* è implementata dalla funzione worker `augment_chunk`. Ogni processo:

- riceve in input un gruppo di indici di immagini,
- costruisce una copia locale della pipeline di Albumentations,
- applica la pipeline a ciascuna immagine del gruppo,
- mantiene un contatore delle immagini elaborate,
- registra informazioni di profiling (PID, tempo di inizio e fine).

Un aspetto delicato riguarda **ciò che il worker restituisce**. Nel benchmark di calcolo ogni worker non restituisce l'immagine augmentata ma una semplice riduzione scalare (la

somma dei suoi pixel). La ragione è metodologica: se i worker restituissero le immagini complete, il benchmark misurerebbe in larga parte il costo di serializzazione e trasferimento attraverso i canali di comunicazione, e non il costo effettivo dell'augmentation. Restituendo un valore scalare, invece, si garantisce che il lavoro venga davvero svolto senza contaminare la misura del tempo di calcolo con l'overhead di comunicazione, che viene analizzato a parte.

Fase di Reduce

Al termine dell'esecuzione parallela, i risultati parziali prodotti dai worker vengono raccolti e combinati. Questa fase avviene nel processo principale, in cui viene fatta un'aggregazione del numero totale di immagini elaborate e la fusione dei risultati parziali in una struttura globale. Poiché nel benchmark di calcolo i worker restituiscono soltanto valori scalari, questa fase è estremamente leggera e non introduce un collo di bottiglia; quando invece si sceglie di restituire le immagini complete, come nel benchmark end-to-end, il worker le scrive direttamente su disco, evitando così che la fase di reduce diventi seriale e congestioni il processo principale.

Strategie di scheduling

- **Static scheduling:** il carico di lavoro viene suddiviso a priori in tanti blocchi quanti sono i processi. Ogni processo riceve un insieme predefinito e contiguo di immagini. Questa strategia riduce l'overhead di scheduling ma può portare a squilibri di carico quando le immagini hanno costi diversi.
- **Dynamic scheduling:** le immagini vengono raggruppate in molti chunk più piccoli che i processi prelevano dinamicamente. I processi richiedono nuovi task non appena terminano quelli assegnati. Questo approccio migliora il bilanciamento del carico, ma può introdurre un overhead maggiore dovuto alla gestione più frequente delle assegnazioni.

Gestione del chunk size

La scelta della dimensione dei *chunk* rappresenta un aspetto critico. Una granularità troppo fine migliora il bilanciamento ma aumenta l'overhead di scheduling, perché ogni task comporta una richiesta e una restituzione del risultato; al contrario, una granularità più grossolana riduce l'overhead ma può causare l'inattività di alcuni processi nella fase finale dell'elaborazione, quando i task ancora in corso sono meno dei worker disponibili. Come si vedrà nell'analisi, per questo specifico carico di lavoro il compromesso è nettamente asimmetrico, e conviene sempre mantenere alcune decine di task per worker.

Verifica di correttezza

Prima di valutare le prestazioni, è stata verificata l'equivalenza tra la versione parallela e quella sequenziale. Su 240 immagini, i risultati prodotti sono stati confrontati tramite SHA-256 in sette diverse configurazioni di parallelismo. In tutti i casi, ogni immagine è stata elaborata una sola volta e il risultato è risultato identico, byte per byte, a quello della versione sequenziale.

Risultati e Analisi

Setup Sperimentale

Setup

Specifiche hardware

Gli esperimenti sono stati eseguiti su una macchina con la seguente configurazione:

- **CPU:** Intel Core i7-1280P (12th Generation)
- **Core:** 14 core fisici, 20 thread (Hyper-Threading)
- **Frequenza:** fino a 4.8 GHz
- **Cache:**
 - L1: 544 KB (data), 704 KB (instruction)
 - L2: 11.5 MB
 - L3: 24 MB
- **Memoria RAM:** 16 GB
- **Sistema operativo:** Ubuntu 24.04.2 LTS

Le misurazioni sono state effettuate con una metodologia rigorosa: ogni configurazione è preceduta da alcune esecuzioni di riscaldamento (*warm-up*), scartate per portare la CPU in regime termico stabile, ed è poi ripetuta 7 volte, riportando la mediana come valore rappresentativo perché meno sensibile agli outlier del sistema operativo. Gli esperimenti sono stati condotti uno alla volta, senza altre applicazioni attive, variando i parametri fondamentali:

- il numero di processi *worker* impiegati, scalando da 1 fino a 40 per osservare anche il comportamento oltre i 20 thread logici;
- il costo della pipeline di augmentation (light, medium, heavy);
- la dimensione dei chunk, la politica di scheduling e il tipo di backend (processi o thread).

Confronto prestazioni tra sequenziale e parallelo

Il primo risultato riguarda il guadagno assoluto della parallelizzazione. Come illustrato in Figura 1, la riduzione dei tempi è drastica soprattutto sui carichi computazionalmente pesanti. Testando la pipeline *heavy* sull'intero dataset di 3000 immagini, il tempo passa dai 19.16s della versione sequenziale a soli 3.74s sfruttando l'infrastruttura parallela, con un throughput che sale da 157 a 803 immagini al secondo. Anche sul carico intermedio la parallelizzazione abbatte il tempo da 3.33s a circa 0.64s. La deviazione standard delle misure resta molto contenuta (dell'ordine dello 0.4% sulla baseline pesante), a conferma della stabilità delle condizioni sperimentali.

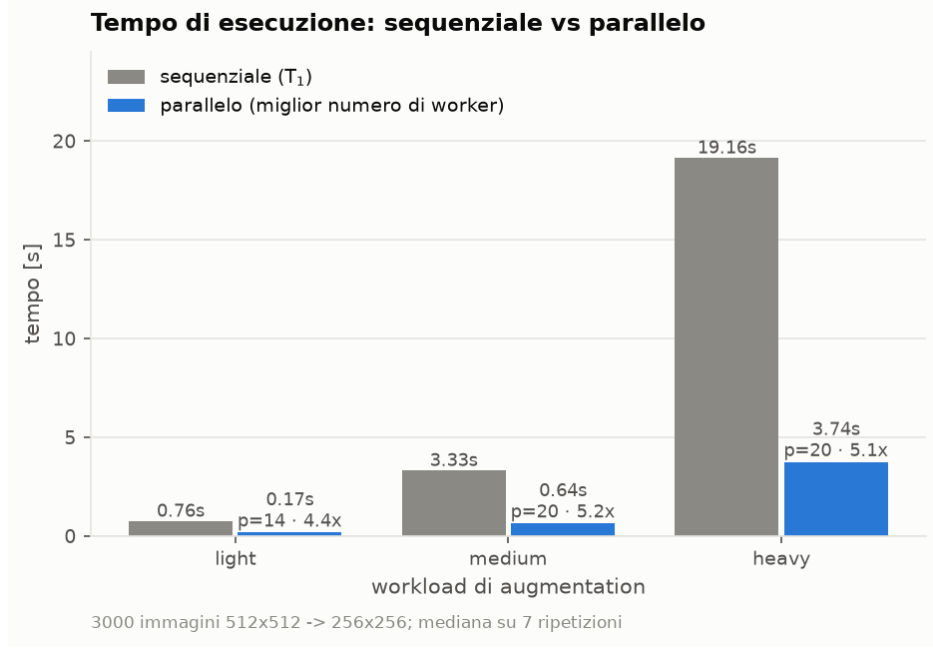


Figure 1: Confronto dei tempi di esecuzione tra la versione sequenziale e quella parallela, per i tre workload. Le etichette indicano il numero di worker ottimale e lo speedup raggiunto.

Analizzando invece i problemi di dimensione ridotta, il guadagno risulta molto più limitato. Sulla pipeline *light*, dove una singola immagine costa appena 0.25 ms, l'elaborazione dell'intero dataset dura meno della creazione stessa dei processi: in questi scenari il tempo di esecuzione è dominato dall'overhead di avvio, cioè dal tempo impiegato dal sistema operativo per istanziare i nuovi processi e preparare i canali di comunicazione.

Speedup e Scalabilità

Per quantificare l'efficienza architetturale è stata utilizzata la metrica dello *Speedup* (S), definita come il rapporto tra il tempo sequenziale e quello parallelo. Il valore di picco è stato ottenuto sul workload intermedio con 20 worker:

$$S = \frac{3.33}{0.635} \approx 5.24 \quad (1)$$

Questo valore sale ulteriormente a $6.19\times$ quando i processi vengono riutilizzati tra più esecuzioni (evitando di ripagare ogni volta il costo di creazione del pool), e a $6.12\times$ sulla pipeline completa da lettura a scrittura sul disco.

L'andamento dello Speedup, riportato in Figura 2, rivela una scalabilità iniziale di tipo quasi lineare fino all'impiego di 4 core. Estendendo il grado di parallelismo fino a 12–16 processi, il sistema continua a registrare un incremento prestazionale, seppur con un'efficienza decrescente. Oltre la soglia dei 20 core logici la curva entra in una chiara fase di saturazione, dove i benefici apportati da ulteriori unità di calcolo si riducono drasticamente, fino a un vero e proprio peggioramento dovuto all'*oversubscription*. La distanza dalla retta ideale non è però un difetto del software: come sarà dimostrato, riflette i limiti fisici della macchina.

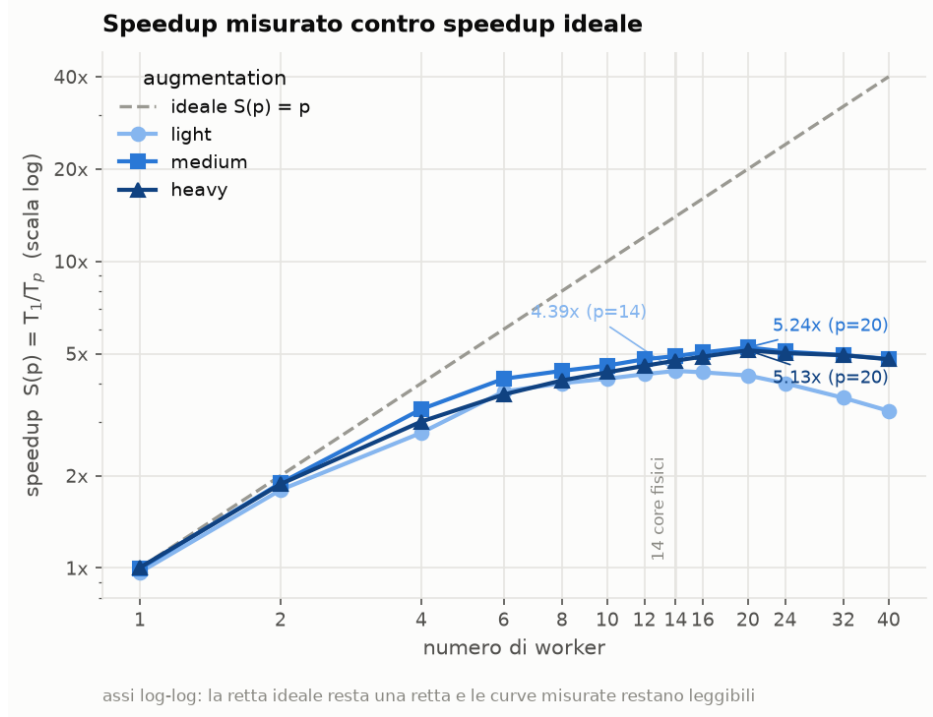


Figure 2: Analisi dello *Speedup* al variare del numero di processi (assi logaritmici). Si osserva un andamento iniziale quasi lineare, seguito da un progressivo appiattimento causato dall'overhead e dai limiti fisici dell'hardware.

Confronto sulle Politiche di Scheduling

Un'altro esperimento riguarda il confronto tra le politiche di scheduling *static* e *dynamic* su un dataset volutamente eterogeneo, in cui le immagini hanno dimensioni molto diverse (e quindi costi che variano fino a un fattore 19). Per questo esperimento è stato usato un indice di sbilanciamento, definito come il rapporto tra il tempo di lavoro del processo più carico e la media: vale 1.0 con distribuzione perfetta e cresce quanto più il carico è squilibrato.

Assegnamento statico contro assegnamento dinamico su dataset eterogeneo

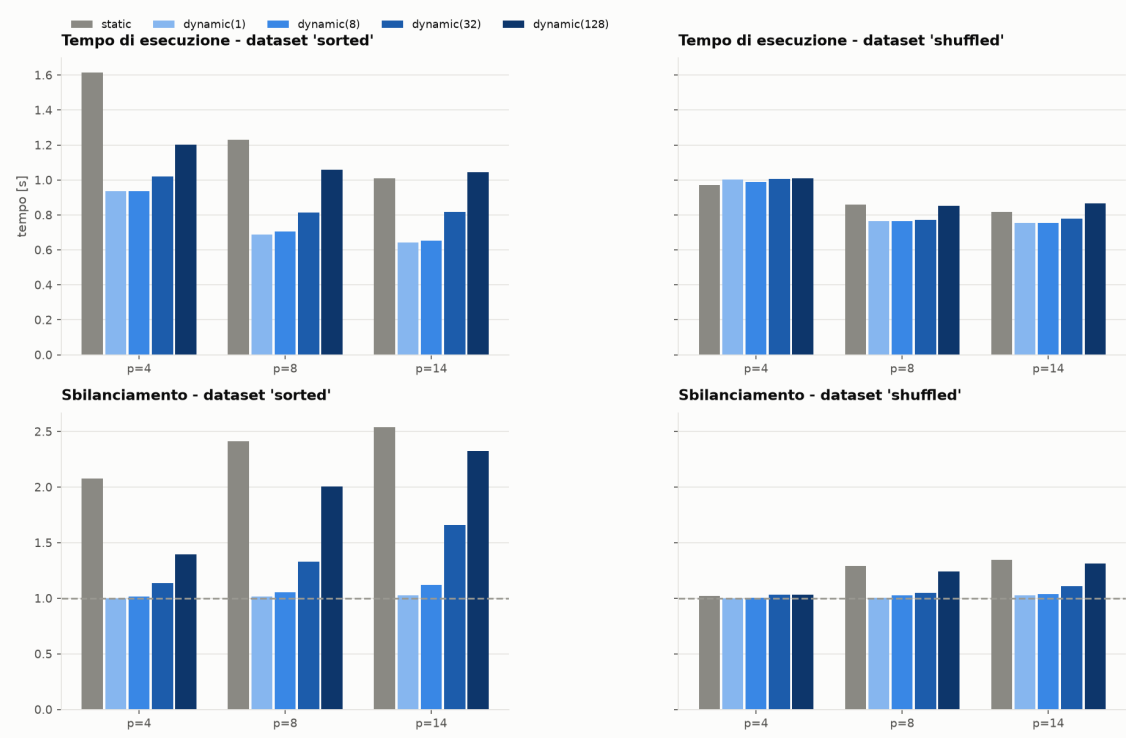


Figure 3: Assegnamento statico contro dinamico su dataset eterogeneo: in alto il tempo di esecuzione, in basso l'indice di sbilanciamento. Il divario è massimo quando le immagini costose sono raggruppate (dataset ordinato).

Come si vede in Figura 3, quando le immagini costose sono concentrate in una porzione del dataset, la politica statica soffre pesantemente: con 14 processi il suo indice di sbilanciamento sale a 2.54, cioè il processo più carico lavora due volte e mezzo la media, mentre gli altri restano inattivi ad aspettare. In questo scenario la scheduling dinamica ha registrato un tempo di 0.642s contro 1.010s di quella statica, con un vantaggio del 57%; su altre configurazioni il divario arriva fino al 78%. Questo comportamento si spiega con la natura asincrona dello scheduling dinamico che, allocando pochi task alla volta, previene l'inattività dei core e garantisce un bilanciamento del carico ottimale. Su carichi uniformi le due strategie si equivalgono, ma la dinamica ha il vantaggio di essere robusta senza richiedere alcuna conoscenza a priori della distribuzione dei costi.

Analisi dello Strong Scaling

Per valutare la capacità di abbattere i tempi di calcolo aggiungendo risorse computazionali, è stata condotta un'analisi di *Strong Scaling*. In questo test la dimensione del problema globale viene mantenuta costante (3000 immagini), mentre si osserva la variazione del tempo di completamento all'aumentare dei processi impiegati.

Dai dati sperimentali si osserva che il tempo di esecuzione subisce un abbattimento drastico e quasi proporzionale nelle fasi iniziali della parallelizzazione (da 1 a 4 core). In questo primo regime l'architettura sfrutta in modo quasi ottimale le risorse aggiuntive. Tuttavia, estendendo ulteriormente il grado di parallelismo, la curva abbandona l'andamento lineare per assumere un comportamento asintotico. Questo fenomeno di saturazione è ben visibile

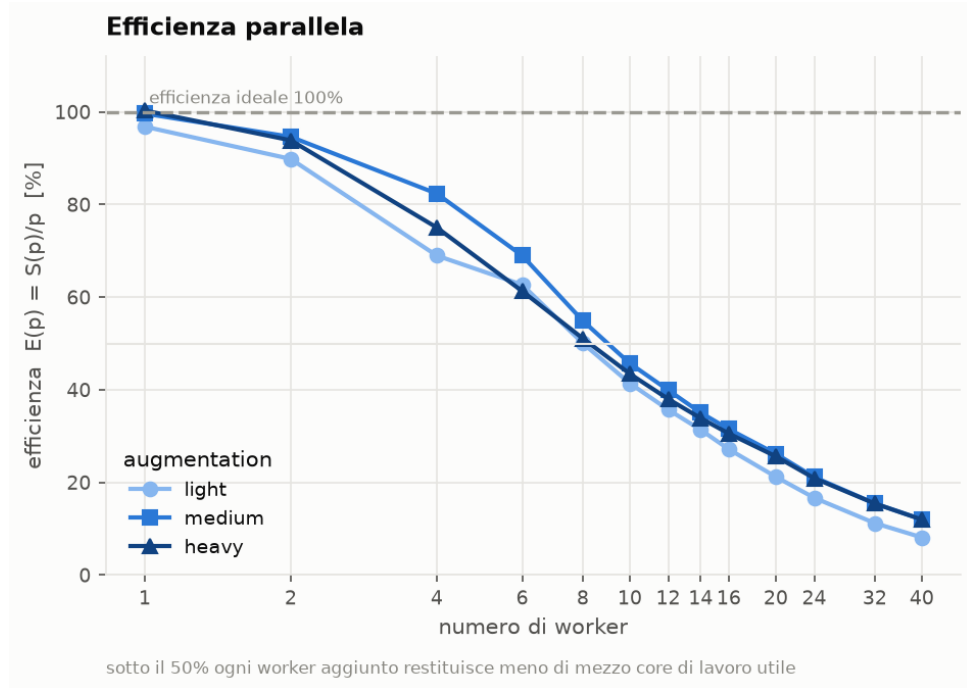


Figure 4: Efficienza parallela $E(p) = S(p)/p$ al variare del numero di processi. La soglia del 50% viene attraversata attorno agli 8 worker per tutti e tre i workload.

osservando l'efficienza (Figura 4), cioè lo speedup rapportato al numero di processi: già a 8 worker essa scende sotto il 50%, e oltre i 20 worker sotto il 25%. In altre parole, ogni processo aggiunto oltre un certo punto restituisce sempre meno lavoro utile.

Weak Scaling

L'analisi di *Weak Scaling* permette di valutare la capacità del sistema di gestire carichi di lavoro crescenti in modo proporzionale alle risorse computazionali impiegate. In questo scenario, la dimensione del problema per singolo processo viene mantenuta costante (fissata a 500 immagini per core), con l'obiettivo teorico di mantenere un tempo di esecuzione invariato all'aumentare del numero di core.

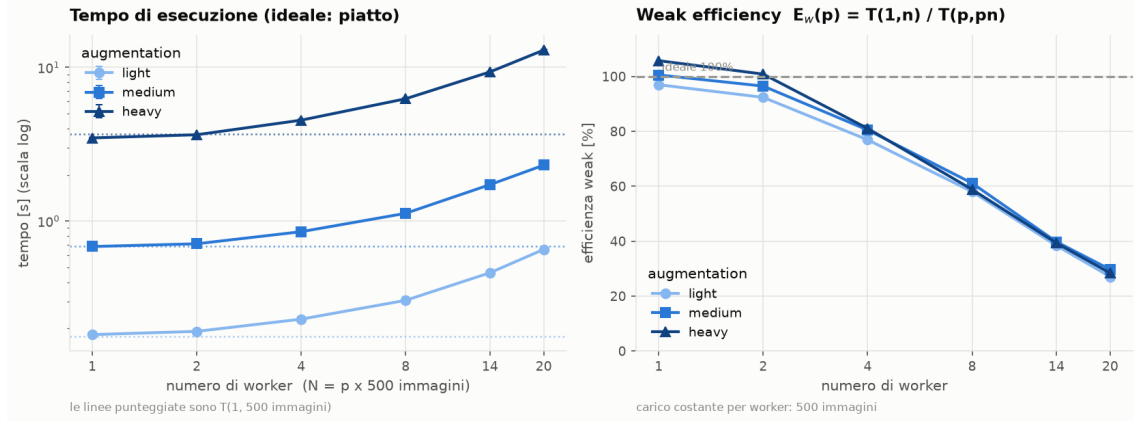


Figure 5: Analisi di *Weak Scaling*: tempo di esecuzione ed efficienza al variare del numero di processi, con carico costante per core (500 immagini). L'allontanamento dall'andamento orizzontale ideale evidenzia la saturazione delle risorse fisiche.

Dai risultati riportati in Figura 5 si osserva che il tempo di esecuzione non rimane costante come previsto dal modello ideale, ma tende a crescere progressivamente. Nel passaggio da 1 a 2 processi l'incremento è quasi trascurabile (efficienza superiore al 100%), mentre oltre la soglia dei 4 processi il sistema mostra i primi segnali di degradazione, fino a un'efficienza del 28% con 20 core. Il confronto con lo strong scaling è particolarmente istruttivo: a 20 worker sul workload pesante l'efficienza weak vale il 28.4%, contro il 25.6% dell'efficienza strong nello stesso punto. I due valori quasi coincidono, e questo è significativo: nello strong scaling ci si aspetterebbe un degrado più marcato, perché la quota di lavoro per worker si assottiglia; nel weak scaling questo non accade, perché ogni worker mantiene lo stesso carico. Il fatto che le due efficienze coincidano dimostra che il degrado non dipende dalla quantità di lavoro per worker, ma semplicemente dal numero di core attivi contemporaneamente.

Analisi del chunk size

L'ultimo parametro analizzato riguarda la granularità della distribuzione dei task, ovvero il chunk size, cioè il numero di immagini inviate a ciascun processo in un singolo pacchetto di lavoro.

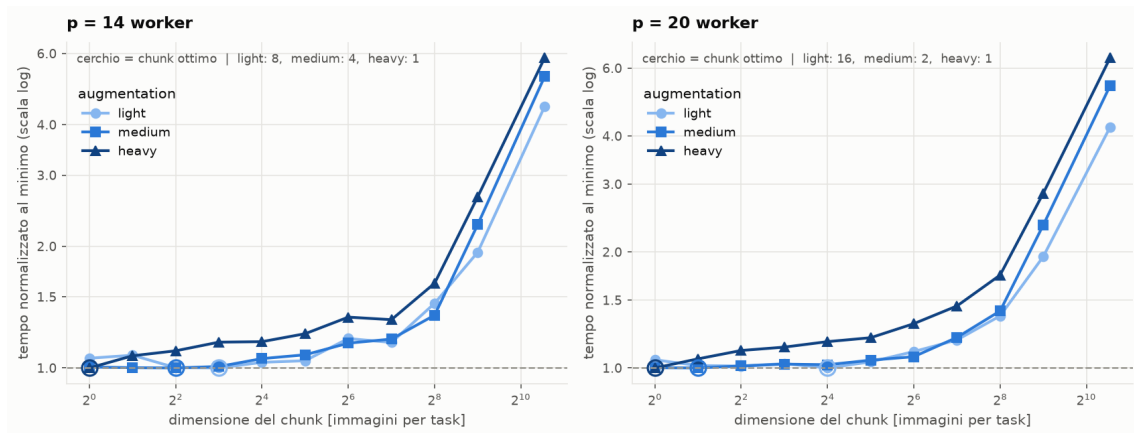


Figure 6: Andamento del tempo di esecuzione al variare della dimensione del *chunk* (normalizzato al minimo di ciascuna curva, assi logaritmici). Si osserva un degrado prestazionale critico all'aumentare della granularità dei task.

I risultati riportati in Figura 6 evidenziano un fenomeno di particolare rilievo, e in parte controintuitivo: il compromesso teorico esiste, ma i suoi due bracci non pesano allo stesso modo. Sbagliare per difetto (chunk troppo piccoli) costa al massimo il 5.7%, mentre sbagliare per eccesso (chunk troppo grandi) fa crescere il tempo fino a cinque volte quello ottimale.

- Con **chunk molto piccoli**, il sistema ottiene prestazioni quasi ottimali, beneficiando di un bilanciamento del carico perfetto. Il costo di un singolo dispatch (poche decine di microsecondi) è talmente piccolo rispetto al lavoro di un'immagine da essere quasi sempre trascurabile: sul workload pesante il chunk ottimale è addirittura pari a 1.
- Con **chunk intermedi**, il tempo inizia a salire gradualmente, segno che la rigidità nell'assegnazione dei blocchi comincia a creare piccoli sbilanciamenti.
- Con **chunk massimi** (l'intero dataset in un solo task), il tempo di esecuzione degenera fino a un valore prossimo a quello della versione sequenziale, perché il parallelismo scompare del tutto.

Quando il chunk size è troppo elevato rispetto al numero totale di task, il sistema perde la sua natura dinamica: i processi ricevono blocchi massicci e indivisibili, e se un processo termina il proprio blocco in anticipo non può aiutare i processi ancora impegnati, lasciando la CPU parzialmente inattiva nelle fasi finali. La regola pratica che emerge è quindi quella di garantire sempre alcune decine di task per worker.

Analisi del Throughput

Per valutare la capacità del sistema di erogare potenza computazionale al netto dei tempi di esecuzione assoluti, la metrica di valutazione è stata spostata sul *Throughput*, definito come il numero di immagini elaborate nell'unità di tempo (immagini al secondo).

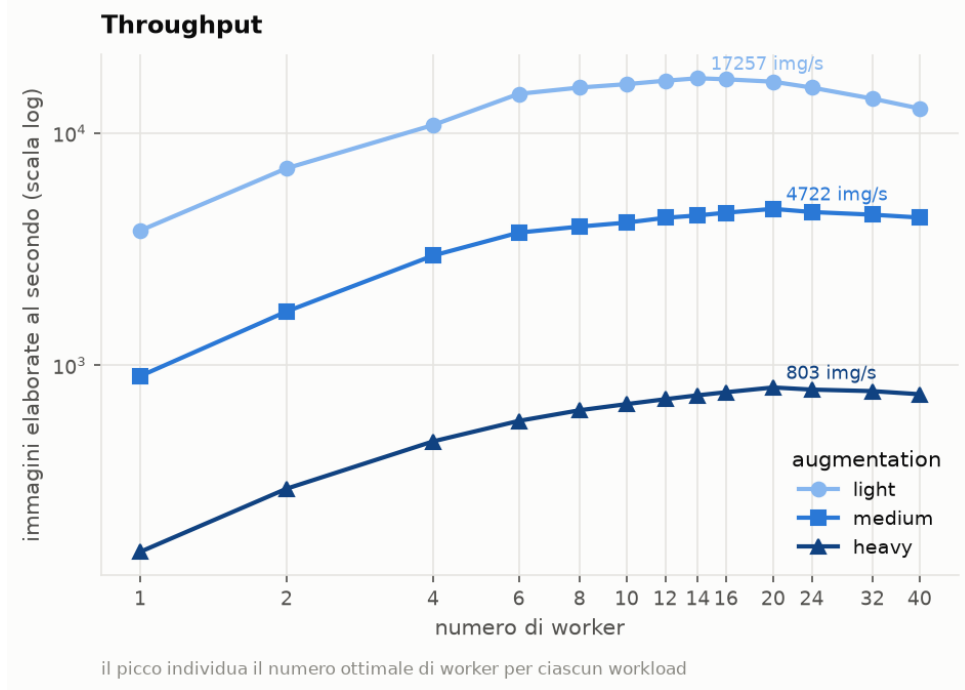


Figure 7: Andamento del Throughput (immagini al secondo, scala logaritmica) al variare dei processi attivi, per i tre workload. Il picco individua il numero ottimale di worker.

Come illustrato in Figura 7, il throughput di picco dipende fortemente dal peso della pipeline: si va dalle circa 17.000 immagini al secondo del workload leggero alle sole 803 di quello pesante, che pur essendo il valore più basso è anche quello che scala meglio. Con la pipeline pesante il throughput continua a crescere fino a 20 worker, perché ogni immagine dà ai processi abbastanza lavoro da ammortizzare i costi di gestione; con quella leggera, invece, la curva si appiattisce molto prima, perché il costo di distribuire e raccogliere i task diventa confrontabile con il tempo di calcolo di una singola immagine. Questa stessa lettura spiega perché la pipeline completa da disco a disco (che include la decodifica e la ricodifica JPEG, entrambe operazioni pesanti ma parallelizzabili) scali addirittura meglio del solo calcolo, raggiungendo il valore di speedup più alto dell'intero progetto in ambito realistico, $6.12\times$.

Conclusioni

Gli overhead introdotti dal software non risultano sufficienti a spiegare il limite osservato nello speedup, che tende a stabilizzarsi intorno a $5\times-6\times$ nonostante la disponibilità di 14 core fisici. Le misure effettuate mediante affinità di processo evidenziano infatti che una parte significativa di tale limitazione è riconducibile direttamente alle caratteristiche dell'architettura hardware.

In particolare, i core del sistema non presentano prestazioni equivalenti. A parità di carico di lavoro, un P-core completa l'esecuzione in 10.22 s, mentre un E-core richiede 16.06 s. L'E-core raggiunge quindi circa il 64% delle prestazioni di un P-core. Ne consegue che il numero di thread logici esposto dal sistema operativo non rappresenta direttamente il numero di unità di calcolo equivalenti disponibili.

L'impiego di un numero crescente di worker non garantisce pertanto un incremento pro-

porzionale delle prestazioni. I worker possono infatti essere distribuiti su core caratterizzati da capacità di calcolo differenti, rendendo il parallelismo effettivo inferiore a quello suggerito dal solo numero di thread disponibili. Questa eterogeneità contribuisce quindi in modo significativo alla saturazione dello speedup osservata all'aumentare del numero di worker.